



图 1: 视频原始封面以“模型提出、系统框架提交”为视觉主线，确立全文对 Agent 生产可靠性的讨论范围。

Source (source_timestamp): 00:00:00–00:00:16

1 总论：生产故障应先审计 Agent 系统框架（Agent harness）

1.1 动机：最危险的故障会呈现为成功

Agent 在生产环境里频繁“翻车”，直觉常把原因归到模型能力：是否理解错了、推理弱了，或提示词不够精确。演讲给出的首要诊断结论更接近分布式系统工程：许多事故发生在模型之外，破坏的是状态、顺序、生命周期、权限或证据边界。读者因此要先学会识别故障形状，再决定是否需优化模型。

开场事故把这种错位暴露得很清楚。用户要求 Agent 记住一项客户退款事实，界面正常显示“已记录”，下一轮却无法恢复这项事实。00:00:32–00:01:50 的叙述强调：消息送达成功，持久记录仍可能留下空洞。崩溃会给出错误和停止点；静默成功会让用户相信结果已经成立，也让运维人员失去显眼告警。模型下一轮仍能流畅回答，只是它在残缺历史上保持了语言连贯。

00:02:03–00:02:39 交代了讲者与案例边界。讲者 Binod 在 OpenAI 从事核心数据与 AI 基础设施工作，此前曾在 Apple 和 Uber 参与分布式系统，也通过 Agent Stack 介绍生产 AI 与数据系统的底层机制。他明确说明，本次演讲是一场系统设计讨论，OpenClaw 只作为公开案例：其 issue、代码与文档让包围 Agent 的系统框架更容易被观察和分析。这一定位意味着后续结论面向通用生产架构，读者无需把它理解成单一产品的功能说明。

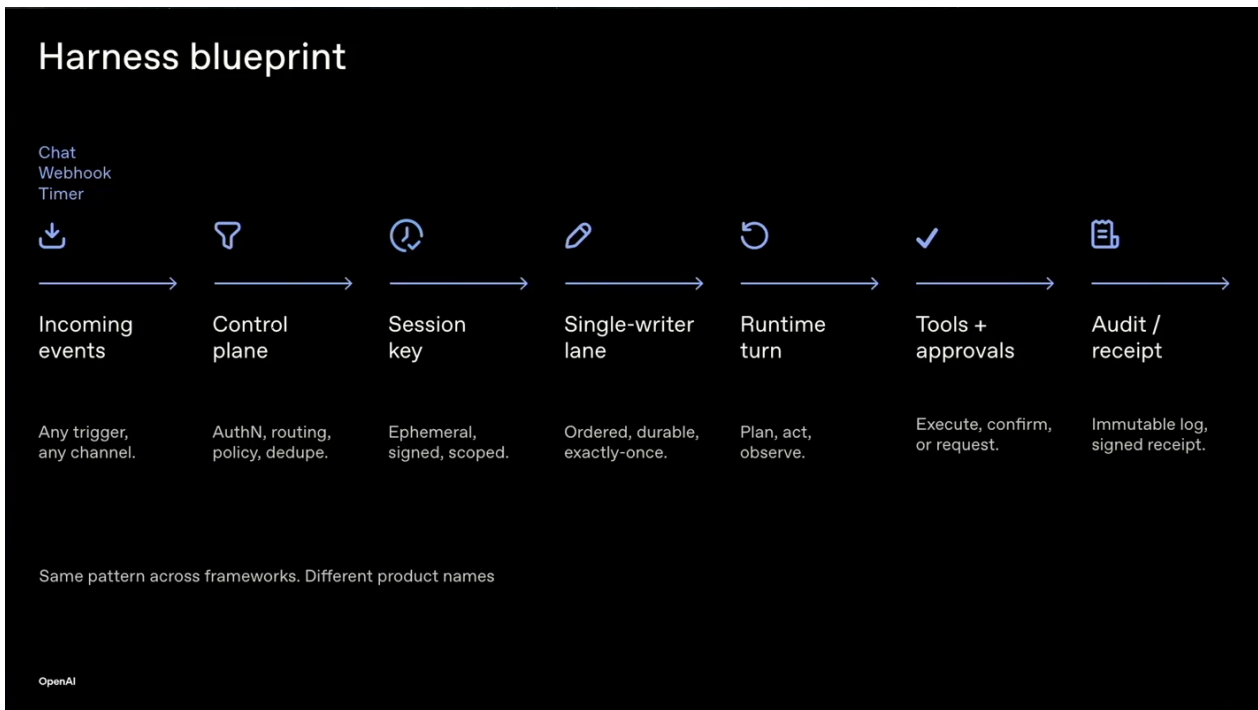


图 2: Agent 系统框架蓝图把多入口事件依次映射到控制面、会话键、单写入通道、运行时、工具审批与执行凭证。

Source (source_timestamp): 00:02:39–00:05:08

核心结论

判断 Agent 是否可靠，要审计包围模型的系统框架。系统框架负责把模型的建议变成受控状态变化，并留下可跨轮次核验的证据。演讲的生产契约是：“模型提出方案，系统框架提交变更，执行凭证（receipt）证明结果。”

1.2 主张：提议、提交与证明属于三个责任主体

模型可以提出消息、工具调用、文件编辑或命令。提议表达意图，尚未改变生产事实。系统框架检查状态边界与执行权限，选择允许的提交路径，随后才可能产生副作用。执行凭证记录系统允许了什么、尝试了什么、执行结果如何，以及用户可见边缘是否确认。三个环节分工清楚，问题定位便能从一句“Agent 说成功了”推进到具体责任边界。

原声片段

演讲者（00:02:39–00:03:07）：“A model proposes, the harness commits, and the receipt proves it.” 这句话把一次自然语言交互拆成意图、状态转换和持久证据三层。

系统框架的运行蓝图从事件开始：聊天消息、Webhook、定时器、心跳或外部系统都可能唤醒 Agent；控制面把事件映射到会话键，再由会话通道约束共享状态的写入顺序；运行时调用模型与工具；策略和审批控制动作；审计轨迹汇总为执行凭证。00:04:16–00:05:08 给出了这条链。汽车类比进一步降低理解门槛：模型像发动机，决定动力；系统框架像方向盘、刹车、交通规则、仪表盘和

黑匣子，决定动力怎样安全地进入现实。高算力只能增加可做之事，生产控制还需要边界与反馈。

1.3 机制：每轮可见材料由系统重新组装

Agent 的连续感来自系统框架在每轮构造工作集（working set）。工作集通常包含对话记录、会话状态、记忆、策略和工具定义；形成它的过程称为上下文组装。模型只能依据当前被提供的材料推理。任何输入缺失或过期，答案依旧可能语法完整、逻辑顺滑。00:05:08–00:06:18 因此提醒：连贯性只能说明模型能处理眼前材料，无法证明材料完整，也无法证明外部动作已经生效。

这套观点并未发明全新的故障类别。超时、重试、幂等、日志、顺序和状态所有权都是成熟工程问题。Agent 场景把概率规划器、多事件入口、逐轮上下文重建和更多动作表面叠加在一起，使旧问题更容易触发、更难从对话外观解释。

1.4 工程检查：建立五个边界的诊断地图

面对一次“模型说已经完成”的运行，工程团队可按五个边界追问：状态由谁持久拥有，下一轮如何恢复；共享变更由哪条路径提交，写入怎样排序；外部工作有哪些截止、取消和终态；执行动作使用了哪一份限定权限；最终哪项用户可见证据被保留下来。每个问题都应落到可命名的组件、记录或命令，模糊回答意味着边界尚未真正建立。

边界与风险

“回答正确”只能评价当前轮次的模型输出。生产可靠性还要求跨轮状态一致、并发变更有序、等待有终点、权限与动作绑定、结果有可重放证据。把这些问题继续塞回提示词，会隐藏真正的系统责任。

1.5 本章小结

本章建立全文总契约：模型负责提出方案，系统框架负责提交受控变更，执行凭证负责证明结果。静默成功事故说明，用户界面健康和持久现实完整是两项独立条件。后续章节将沿“现象—边界—机制—证据—行动”的读者路径，依次展开状态所有权、变更秩序、生命周期、执行权限与证据闭环。

2 状态所有权：能重放的事实才构成可靠记忆

2.1 动机：送达成功与系统记住是两项条件

Agent 回复已经抵达用户界面，通常会被当作整次交互成功。这个判断漏掉了跨轮次系统最重要的问题：未来运行能否从权威记录恢复同一事实。00:06:27–00:07:13 重述开场的状态洞事故：Telegram 回复成功，日志也显得健康，相关 router turn 却没有回到活跃上下文或对话记录。下一轮面对缺失历史，模型仍可生成流畅回答，用户则会感到系统突然失忆。

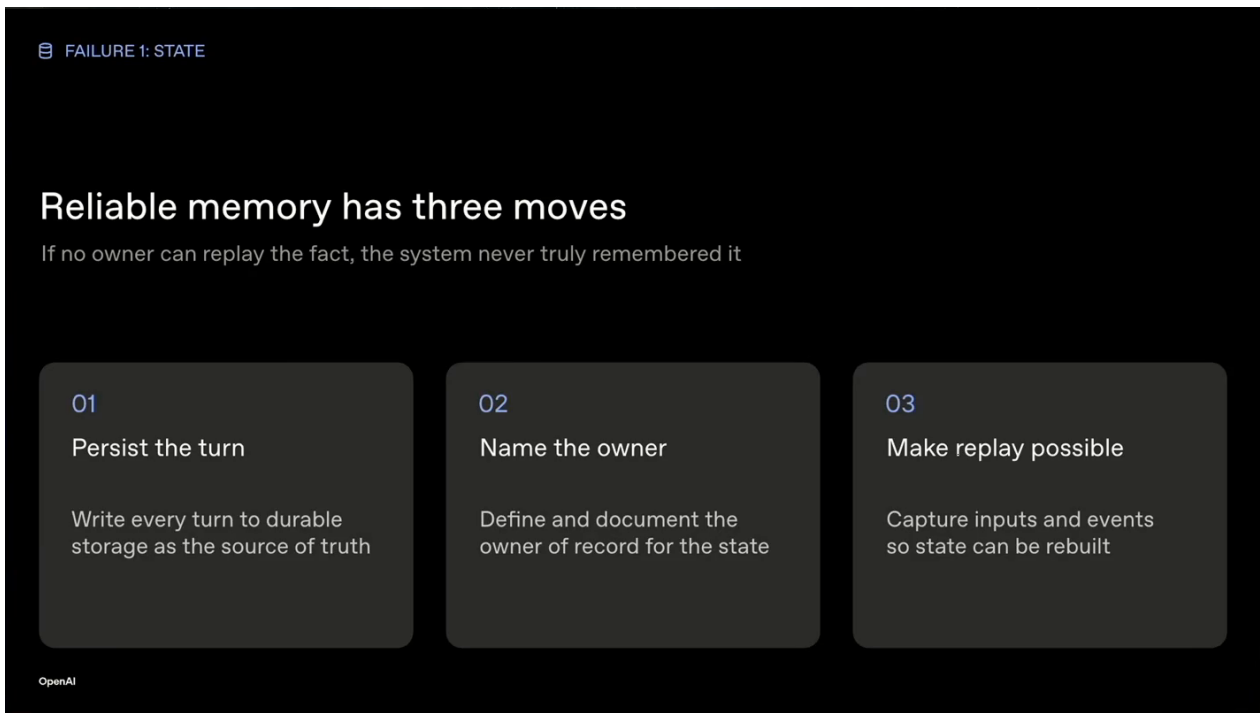


图 3: 可靠记忆需要持久化当前轮次、指定事实所有者并建立重放路径；界面送达无法替代这三项机制。

Source (source_timestamp): 00:06:27–00:08:11

核心结论

一项事实只有在被命名的所有者能够持久保存并确定性重放时，才成为 Agent 的可靠记忆。发送动作证明交付发生过；未来上下文能恢复该事实，才证明系统真正记住了它。

可靠记忆需要一条重放路径（replay path）：后续轮次必须能够从权威记录确定性恢复同一事实。这个路径把用户眼前的交付结果连接到未来运行可继承的持久现实，也为下图两条成功条件提供统一判断标准。

现有图把用户可见回复与持久记录恢复画成两条独立路径。界面这一条路径变绿时，记录恢复仍可能断裂。00:06:27–00:08:11 的案例因此说明“看起来成功”的危险：用户没有理由重试，系统也可能没有错误信号，缺口直到下一轮需要事实时才显现。

2.2 主张：为每类未来事实指定权威所有者

事实源（source of truth）指其持久状态被系统承认为现实依据的权威所有者。记录系统（system of record）强调承担持久记录和重建责任的具体系统。二者关注“哪份状态算数、谁能恢复现实”；普通存储只说明字节放在哪里。一个目录里存在 JSON 文件，并不自动使该目录获得事实所有权。

演讲在 00:07:13–00:07:43 给出多个直观映射：日历事件归日历系统，客服状态归工单系统，代码变更归工作区或代码树，会话轮次归对话记录，用户偏好归记忆存储。映射的价值在于消除“大家都存一点”的模糊地带。若多个组件各自保留副本，却无人能说明冲突时以谁为准，系统仍然没有明确所有者。

2.3 机制：把所有者、提交与恢复连成闭环

重放路径落地时至少连接四个环节：用稳定身份标识事实；由记录系统承担持久所有权；在明确提交点确认写入已经生效；后续上下文组装按同一身份重新加载。系统接收一项“请记住退款”的请求时，应依次处理当前 turn、把退款事实提交给命名记录系统，并在下一轮从该系统恢复。任一环节缺失都会让一句正确回复停留在短暂外观。

背景知识

状态所有权比数据位置更深一层。缓存、日志和消息队列都可以保留字节；权威所有者还要定义冲突裁决、更新规则和恢复语义。工程上应让派生副本能够从事实源重建，而不要让副本暗中变成第二份现实。

这也说明为什么对话记录不能包揽所有事实。对话记录适合证明 Agent 说过什么；日历、工单或代码仓库分别掌握自己的业务现实。系统框架应在工作集里引用这些权威状态，避免把自然语言陈述误当作已经完成的外部提交。

2.4 源中例证：静默成功怎样继承残缺现实

开场案例的时间链很具体：00:00:50 左右，用户要求记录客户退款；00:00:58 左右，助手承诺下一轮会记得；到 00:01:06，下一轮已无法重新考虑该事实。用户体验层获得成功，系统继承的现实却不完整。故障点位于持久提交或重新装载边界，和回答措辞是否自然没有直接关系。

边界与风险

健康日志可能只覆盖“发送完成”。若日志没有记录事实所有者、提交身份和后续重放结果，它无法排除状态洞。单次成功响应也不能替代一次跨轮恢复测试。

2.5 工程检查：逐项填写所有权与恢复证据

团队可从真实运行里选出每项“未来可能再次使用”的事实，并回答四个问题：它的稳定身份是什么；哪个记录系统拥有它；写入何时构成提交；下一轮通过哪条接口恢复。随后执行一次受控重放，从空工作集开始，只依赖权威记录重建目标事实。若恢复需要猜测临时文件、搜索日志或依赖原进程内存，所有权设计仍不完整。

检查还应覆盖失败语义：持久提交失败时，界面能否继续宣称“已记住”；重放数据过期时，系统是否标明版本；多个副本冲突时，哪一方裁决。清晰的失败会阻止静默成功进入下一轮。

2.6 本章小结

可靠记忆由“命名所有者+重放路径”共同构成。事实源决定哪份持久状态算数，记录系统承担具体保存与重建责任，重放路径让后续轮次确定性恢复。下一章将在所有者已经明确的基础上继续追问：当多个执行者同时修改同一事实边界时，提交应怎样排序。

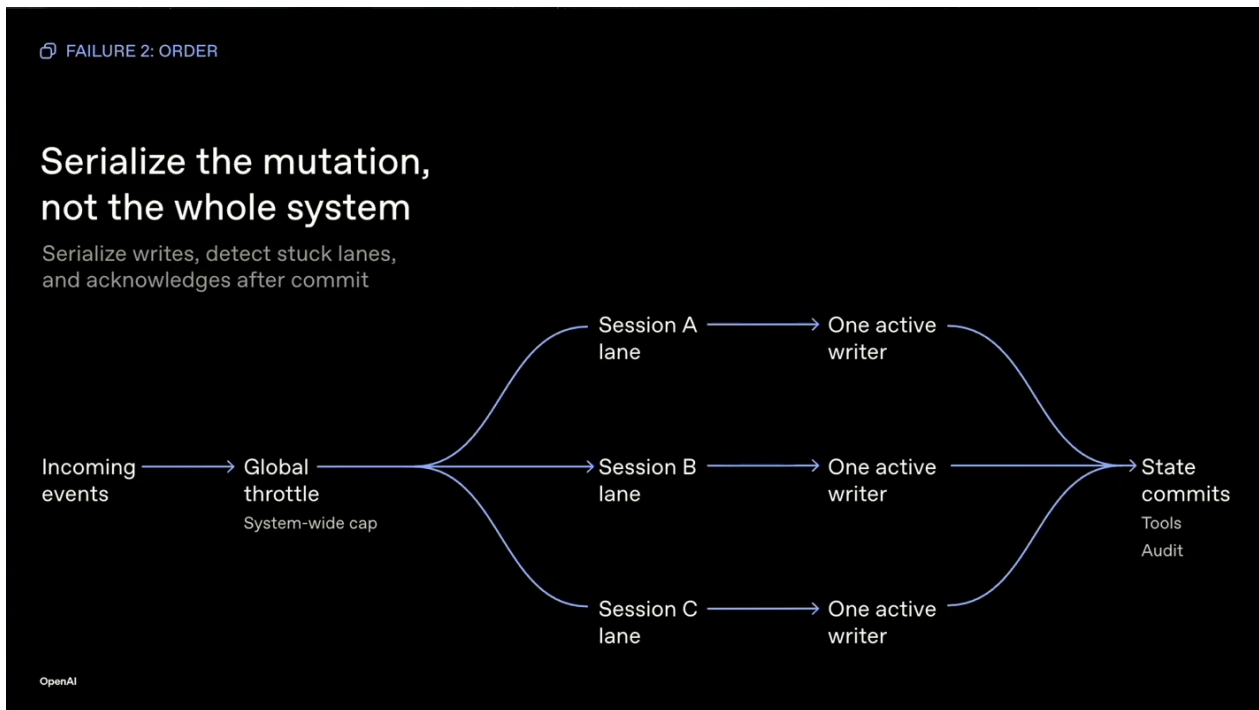


图 4: 多个会话可以并行运行，每个可变状态边界仍由一个活跃写入者收敛到受审计的状态提交路径。

Source (source_timestamp): 00:08:18–00:09:35

3 变更秩序：共享可变边界需要单一提交路径

3.1 动机：两个局部正确的写入也会产生全局错误

确定了事实所有者，只解决“谁的状态算数”。生产系统还必须回答“谁可以改变它，以及改变按什么顺序生效”。00:08:18–00:08:58 描述了一次读取—修改—保存竞争（load-modify-save race）：两个调用者读到同一份旧状态，各自修改不同记录，随后分别保存；第二次保存基于旧快照，静默覆盖第一次结果。两个写入者的局部逻辑都正确，组合结果却丢失了已经完成的变更。

用户看到的症状具有强烈的人格错觉：刚撤销的承诺再次出现，已经处理的跟进重复发送，刚做的更正被系统“忘记”。根因位于共享状态的提交顺序。把它解释成模型善变，会让修复继续偏离事故边界。

核心结论

一个共享可变状态边界应收敛到一条受审计提交路径（audit commit path），并在提交临界区保持一个活跃写入者。该约束允许并行读取、独立检索、子 Agent 扇出以及不同会话同时运行。

图中展示全局节流、三个并行会话通道、每条通道一个活跃写入者，以及各通道收敛出的状态提交。00:08:18–00:09:35 的对照直接揭示，风险集中在共享事实发生不可逆改变的窄边界。

3.2 机制：会话键确定身份，会话通道约束顺序

事件可能来自聊天、Webhook、定时器或工具回调。系统框架先用会话键（**session key**）把这些入口映射到同一状态身份，再用会话通道（**session lane**）约束该边界内的活跃写入与执行顺序。会话键回答“这些事件是否属于同一份状态”，会话通道回答“同一状态中的动作按什么次序提交”。身份映射错误会拆散本应共享的历史；通道缺失会让相同身份遭遇重叠写入。

单一提交路径可以由队列、互斥锁、事务或分布式锁实现。选择机制时应围绕同一不变量：在读取当前权威版本、验证前置条件、写入新版本和记录执行凭证的临界区，只有一个提交者能够成功。临界区之外的推理、检索和候选生成仍可并行，从而保留 **Agent** 系统需要的吞吐。

背景知识

并行与串行承担互补职责。并行适合无副作用的读取、独立搜索和候选方案生成；串行适合共享状态的最终裁决与提交。工程设计可以像数据库事务：多个客户端并发准备，提交点通过版本检查或锁保证唯一有效顺序。

3.3 源中例证：最后写入者获胜会丢失已确认事实

演讲中的覆盖竞争可用一个客服状态例子理解。**Writer A** 从版本 7 读取任务列表并移除一项已完成承诺；**Writer B** 也从版本 7 读取列表并添加新的跟进。**A** 保存版本 8，**B** 随后把自己的完整旧快照保存为另一个结果，于是 **A** 的删除消失。**B** 并未主动撤销 **A**，它只是缺少版本前置条件和排序机制。所谓“最后写入者获胜”只描述覆盖现象，无法提供一致性保证。

00:08:58–00:09:28 明确限定了正确边界：子 **Agent** 扇出、并行读取、独立检索、多会话并发都可以继续；每个可变状态边界的提交必须经过同一受审计路径。锁覆盖整个系统会造成不必要等待；锁定或事务保护提交时段，才能同时获得正确性与吞吐。

3.4 工程检查：验证提交身份、版本与可恢复顺序

检查一条生产写路径时，团队应先确定共享状态边界和会话键，再列出全部可能写入者，包括主 **Agent**、子 **Agent**、回调、重试进程和恢复命令。每个写入都应携带预期版本或等价前置条件，经同一提交接口完成；接口要原子地记录新版本、动作身份和执行凭证。发生版本冲突时，系统应拒绝过期提交，并以最新状态重新计算，避免直接覆盖。

随后进行重叠写验证：让两个写入者读取同一旧版本，分别产生可区分变更，再控制提交顺序。合格结果应表现为一个提交成功，另一个收到稳定冲突并重新基于新版本处理；审计记录能够重建实际顺序。若最终状态取决于线程时机，或日志只能看到两个“**success**”，提交边界仍有缺口。

边界与风险

“一个活跃写入者”约束的是共享状态的提交临界区。将整条 **Agent** 流程全局串行化会放大延迟和堵塞风险，也掩盖了真正需要保护的可变边界。

3.5 本章小结

变更可靠性建立在清晰身份与可审计顺序上：会话键聚合同一状态，会话通道组织边界内的执行，受审计提交路径让最终写入具备唯一顺序和版本证据。并行工作可以保留，副作用提交必须收敛。下一章将进一步处理时间边界，确保每项已经进入通道的外部工作都能抵达明确终态。

4 生命周期 (lifecycle): 让每一次外部工作都有结局

4.1 动机：沉默会把一个故障扩散成整条通道停摆

生产系统最危险的等待，往往没有报错画面。工具调用已经写入会话，匹配的工具结果却没有回来；运行时仍把它当作进行中，后续消息只能排在它后面。用户看到的是 Agent 毫无反应，系统内部看到的则是一项永远等不到事件的工作。此处需要治理的是生命周期：一次外部工作的开始、等待、截止、取消、恢复以及终态记录规则。生命周期若没有闭合条件，队列、重试和人工介入都会失去共同依据。

核心结论

沉默等待不构成终态。每项外部工作都必须落入一个可记录的终态结果 (terminal outcome)：成功、失败、超时、取消，或达到最大尝试次数；终态还要绑定可持久核验的执行凭证。

4.2 主张与机制：先界定时间，再设计恢复

截止时间回答“系统愿意等到何时”，超时把越界等待转换成明确结果，取消回答“谁可以停止工作”，恢复命令则回答“通道被阻塞后怎样重新获得控制”。这四项必须共同设计。仅有超时数值，缺少结果写入时，下一轮仍无法判断工作是否完成；仅有普通取消命令，而取消命令也排在阻塞工作之后，它永远无法执行。

可靠实现可以把一次外部调用看成有限状态推进：已请求、执行中、终态结果。进入执行中时便登记截止时间和尝试上限；进程退出、连接断开或响应超时后，系统写入对应终态并生成执行凭证；恢复命令走独立控制路径，能够终止或隔离卡住的工作，再允许会话通道继续处理新事件。监控需要暴露运行已等待多久、当前尝试次数和最后一次可验证进展，让“卡住”成为可观察状态。

4.3 源中例证：dangling tool call 如何堵住后续消息

演讲在 00:09:57--00:11:19 展示了 dangling tool call：会话里存在工具调用，却没有对应工具结果。可能原因包括进程死亡、连接中断，或超时发生在结果落盘之前。具体原因影响调试方向，生产层面的共同事实更直接——运行正在等待一个不会到来的事件，新消息持续积压，Agent 在用户眼中像是“死掉了”。

讲者给出的闭环包括：运行需要截止时间与取消；工具需要超时和错误结果；看门狗让卡住的工作显形；通道需要一条不会继续排在卡住工作后面的恢复命令。尤其关键的是，执行凭证要记录

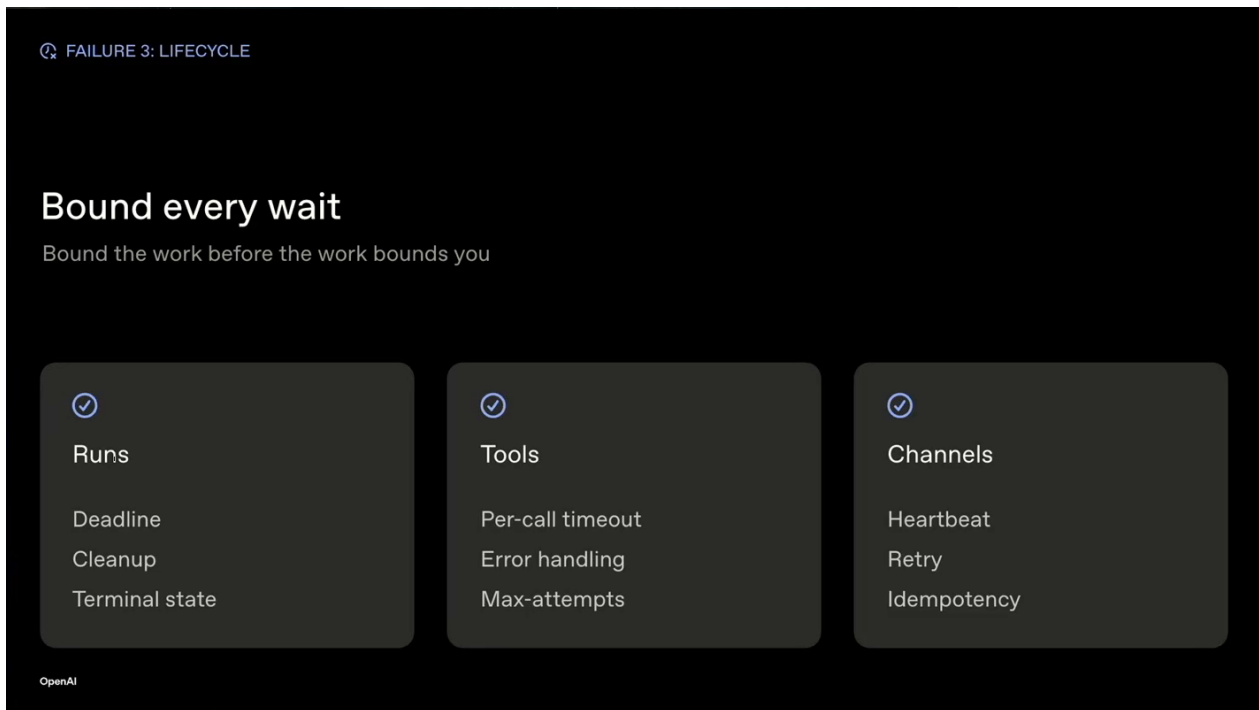


图 5：外部等待必须有界：运行、工具与通道分别以截止、超时、错误处理、最大尝试和恢复机制进入明确终态。

Source (source_timestamp): 00:09:57–00:11:19

最终落入成功、失败、超时、取消或最大尝试次数中的哪一种。下一步据此选择继续、补偿、重试或请求人工处理，无需从沉默中猜测。

边界与风险

超时之后直接重试仍可能制造重复副作用。系统还需把尝试编号、原始动作和终态写进同一执行凭证；涉及外部写入时，应由后续章节所述的权限对象与幂等边界共同约束重试。

4.4 工程检查：逐项审计外部边界

对网络请求、工具进程、子 Agent、队列任务和人工审批，可执行以下检查：

检查对象应是一条真实运行记录。设计文档中的超时声明只能证明预期，运行记录中的截止时间、终态与资源释放才能证明机制确实生效。团队还应选择一次进程中断进行恢复演练，确认重启后的判断来自持久证据。

1. 是否在创建工作时写入明确截止时间、最大尝试次数和责任主体；
2. 是否把无响应转换成超时结果，并将错误原因与尝试编号持久化；
3. 取消是否可重复调用，且能够留下“谁在何时取消了什么”的执行凭证；
4. 恢复命令是否拥有独立控制路径，能够绕开正在阻塞的会话通道；
5. 进程重启后，系统能否从记录判断工作已终止、可安全重试或需人工确认；

6. 每个终态是否会释放租约、锁或通道占用, 使后续消息获得确定进展。

4.5 本章小结

生命周期治理把“还在等”变成有边界、可观察、可恢复的生产状态。截止时间限制等待, 超时与取消形成明确终态, 恢复命令重新取得控制, 执行凭证让后续轮次知道真实结局。讲者的简明提醒是: 先约束工作, 避免工作反过来约束整个系统。

5 执行权限 (execution authority): 把批准绑定到一个具体动作

5.1 动机: 对话一旦转化为动作, 风险边界随之改变

模型可以规划发送消息、修改文件或调用外部服务, 这说明它拥有能力 (capability), 即可以提出某类请求。真实副作用还需要执行权限: 允许某一主体在特定会话与运行中, 使用限定工具、参数、范围和有效期执行一个具体动作的授权对象。能力描述可请求的表面, 执行权限决定某次请求能否越过生产边界。两者混在一起时, 一句流畅建议可能被误当成持续有效的授权。

核心结论

模型负责推理和请求, 系统负责权限裁决。一次有效授权必须能回答: 谁、在哪个会话和运行中、使用哪个工具与哪些参数、作用于什么范围、有效多久、最终结果是什么, 以及对应执行凭证在哪里。

5.2 主张与机制: 最小权限、作用域凭据与动作绑定审批

最小权限收窄可用工具与操作集合, 作用域化凭据限制真实执行身份能访问的资源, 作用域化审批 (scoped approval) 则把人的决定固定在一个待执行动作上。三者分别控制“能看见什么工具”“以什么身份行动”和“这一次具体行动是否获准”, 共同形成可审计的权限边界。

权限对象至少应绑定 actor、session、run、tool、arguments、scope、lifetime、outcome 与执行凭证。系统接收回调、重试或重放时, 必须重新核对这些字段仍然指向同一动作, 并检查期限与当前状态。审批已过期、动作参数改变、运行身份变化或原动作已有终态时, 该对象应进入明确终态, 释放通道占用, 并留下拒绝或失效记录。

5.3 源中例证: 过期 callback 造成 approval drift

在 00:11:22--00:13:05, 讲者分析 approval drift: 一个已过期的审批回调被当作可恢复工作, 持久残留的回调状态继续阻塞后续通道任务。界面上曾经存在按钮点击, 当前动作却已经没有有效权限。根因是系统把审批保存成“人曾经靠近系统并点击同意”的模糊记忆, 未持续绑定原始动作。

演讲进一步列出一个有用审批对象的问题: 谁批准、发生在哪个 session 和 run、针对哪个 tool 与 arguments、有效多久、结果怎样, 并指向哪一份执行凭证。任何字段在 retry、replay 或 channel

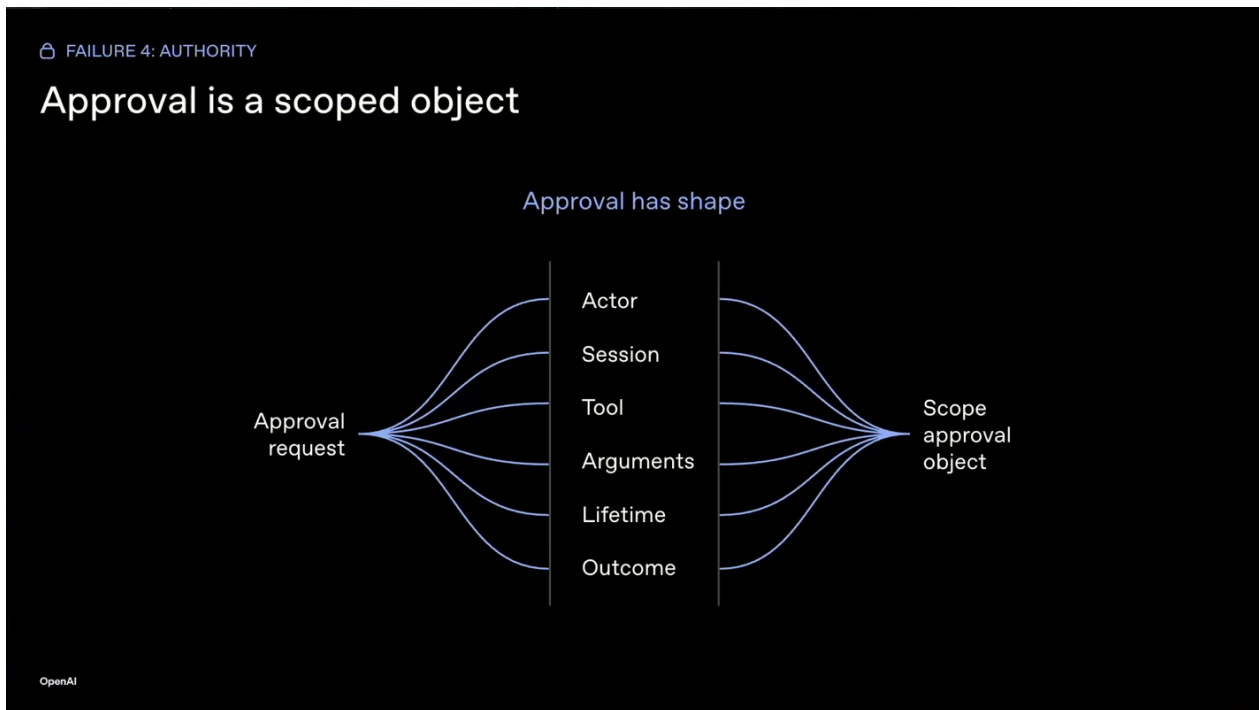


图 6: 作用域化审批把主体、会话、工具、参数、有效期与结果绑定为一个可核验的执行权限对象。
Source (source_timestamp): 00:11:22–00:13:05

callback 中丢失，系统便无法证明当前执行仍是当初获批的动作。过期路径必须终止，不能循环重试；失效回调持续循环，会把权限错误升级成生命周期故障。

背景知识

权限校验应使用结构化动作身份。例如“允许本运行在十分钟内，用工单更新工具把编号 42 的状态改为已解决”具有主体、范围、参数和期限；“用户刚才点过同意”缺少可重放的动作绑定。

5.4 工程检查：审查一次权限对象是否完整

落地时可沿以下顺序检查：

1. 主体：actor、session 与 run 是否明确，回调能否证明仍属于该运行；
2. 动作：tool 和完整 arguments 是否被固定，参数变化是否触发新审批；
3. 范围：资源、目标对象与允许操作是否遵循最小权限；
4. 身份：作用域化凭据是否只服务该动作，是否会被其他任务复用；
5. 时间：审批起止、过期行为与最大重试次数是否明确；
6. 结局：允许、拒绝、过期、执行失败或成功是否均有终态结果；
7. 证据：策略裁决、执行尝试和外部结果是否汇入同一执行凭证。

审计还应专门模拟一次过期回调和一次参数改变后的重试。预期行为是权限校验稳定拒绝旧授权、记录原因并释放通道；若系统继续执行或持续等待，说明动作绑定已经漂移。

另一项实用检查是从执行凭证反向重建审批对象：审计者应能仅凭记录找回批准主体、动作参数、凭据范围、有效期限和最终结果。若只能找到一枚“已批准”布尔值，该记录无法解释批准覆盖了哪次执行，也无法支撑事故后的责任追踪。

5.5 本章小结

执行权限是一份有主体、有动作、有范围、有期限、有结局的生产对象。最小权限收窄工具面，作用域化凭据限定身份，作用域化审批固定人的决定。模型可以理解权限边界并提出请求，最终裁决和可审计提交由系统完成。动作获准并执行以后，下一章还要检查用户边缘证明是否成立，并由执行凭证保存从裁决到结果的完整证据链。

6 证据与审计：把生产成功闭合在用户关心的边缘

6.1 动机：内部 **success** 仍可能对应用户眼中的空白

工具返回 **success**，只能说明某个内部组件接受或完成了请求。用户关心的边缘可能仍无变化：消息没有渲染、工单没有更新、文件没有落盘、日历事件没有出现。此时 **Agent** 若说“已经发送”，用户回答“没有看到”，双方都在陈述各自可见范围内的事实。系统缺少的是用户边缘证明（**edge proof**），即在真正外部表面确认结果存在或明确失败的证据。

核心结论

生产成功是一条证据链：模型提议、策略允许或拒绝、系统执行尝试、用户边缘确认或未确认结果，最后由执行凭证保存整条链。对话记录与单一工具结果都只能覆盖其中一段。

6.2 主张与机制：从 **proposal** 追踪到执行凭证

完整链条从 **proposal** 开始，记录模型希望执行什么；**policy** 保存策略依据和权限裁决；**attempt** 记录真实工具或 **API**、参数、尝试编号与外部响应；用户边缘证明检查用户关心的表面；执行凭证把前述节点、终态结果和相互绑定关系跨轮次保存。涉及重试时，首次出现的幂等键（**idempotency key**）用于识别同一执行意图，避免重复副作用。

演讲在 00:13:05--00:14:10 给出用户边缘证明缺失案例：消息工具对 **Web Chat** 或 **TUI** 报告成功，消息却没有渲染，普通助手回复仍可出现。工具结果证明内部路径接受了请求，没有证明用户看到结果。因而证据必须终止于用户关心的边界；边缘确认失败也应成为明确终态，允许系统采取补偿、重试或人工告警。

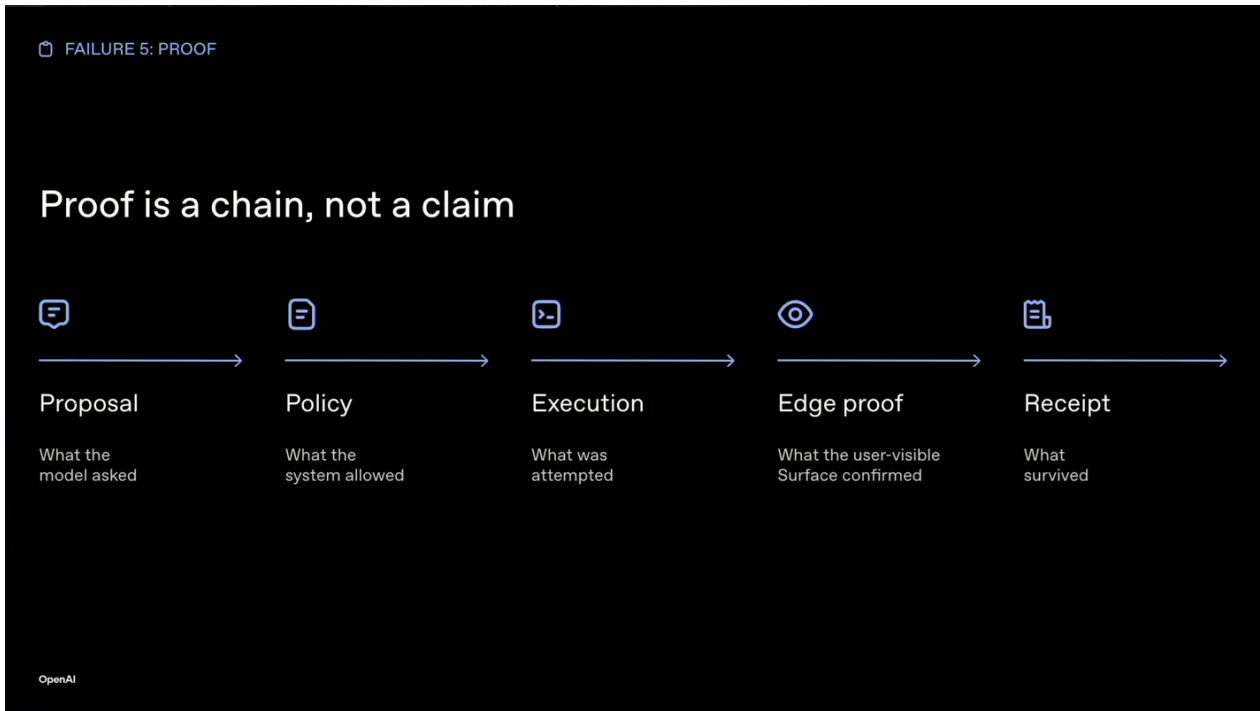


图 7：证明是一条从模型提议、策略裁决、执行尝试、用户边缘证明到执行凭证的完整证据链。

Source (source_timestamp): 00:13:05–00:14:10

6.3 五问审计：把流畅对话还原为可检查运行

讲者在 00:14:10--00:16:55 建议团队只选择一个 Agent 系统和一条真实生产路径，索取其执行凭证，并连续追问五个问题：

1. 什么唤醒了它？记录用户消息、webhook、定时器、工具结果、子 Agent 或重放的触发类型与唯一身份，以检查去重、顺序和授权。
2. 它继承了什么状态？列出对话记录、会话状态、记忆快照、策略版本与工具面。模型只能依据系统框架组装的工作集推理。
3. 它使用了哪项权限？保存 actor、session、run、tool、arguments、scope 与 lifetime，证明授权绑定一个待执行动作。
4. 实际执行了什么？记录工具或 API 调用、完整参数、attempt 编号、幂等键和外部结果，避免用意图摘要替代副作用事实。
5. 什么证据留存下来？核对工单、消息、文件或日历事件在用户边缘的真实状态，并把确认结果写入执行凭证。

这五问能够暴露因果链上的缺口。若触发身份缺失，系统无法判断重复事件；若继承状态不明，流畅回答也可能建立在残缺历史上；若权限字段脱落，执行失去授权依据；若 attempt 只有摘要，副作用无法复核；若证据停在内部 success，用户结果仍未闭环。

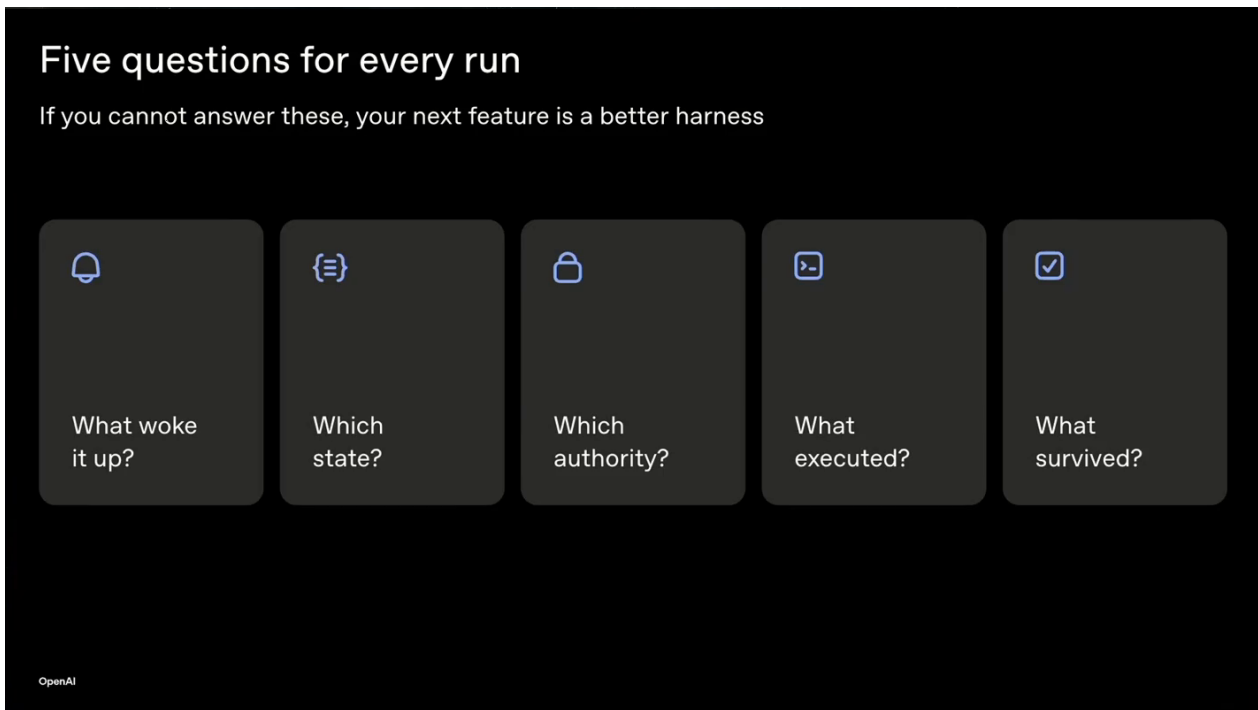


图 8: 一次生产运行的五问审计：触发源、继承状态、执行权限、实际执行与留存证据共同把流畅对话还原为可检查记录。

Source (source_timestamp): 00:14:10–00:16:55

6.4 工程检查：用一次真实路径验证五个边界

团队可以选择“收到退款请求后更新工单并回复用户”这类真实路径，从入口事件开始逐项串联：确认触发事件映射到正确会话键；确认事实源能通过重放路径恢复退款事实；确认共享修改进入受审计提交路径；确认外部调用拥有截止时间和终态；确认执行权限绑定具体工单、参数和期限；最后在工单系统与用户界面核对边缘结果。每个节点都应能指向前一节点的身份和后一节点的结果，任一断链都应让运行保持未证明或失败状态。

把开场事故代入同一检查：用户消息唤醒运行，渠道发送确实执行，交付证据也留存下来；持久会话回合没有进入事实源，下一轮无法重放。交付存活而状态没有存活，这个差距精确定位了系统框架故障。修复重点由“再调一次提示词”转向补齐状态所有权、提交秩序和完整执行凭证。

6.5 讲者收束与总结延伸

演讲在 00:16:55--00:18:05 回到三句主线：拥有状态、排序变更、证明动作。更强模型改善单轮内部推理；跨轮次的系统健康依赖状态所有权、变更秩序、生命周期、执行权限与证据。讲者再次强调主契约：“模型提出方案，系统框架提交变更，执行凭证证明结果”，并邀请听众继续查阅 OpenAI Agents SDK 与 Agent Stack，或会后讨论各自的系统框架设计。

进一步看，五个边界组成一条连续责任链：状态所有权定义哪份现实算数，提交秩序保护现实不被并发覆盖，生命周期保证工作能够结束，执行权限限定谁能造成副作用，执行凭证证明副作用抵达用户边缘。任何单点健康都无法替代整链闭合。团队最有价值的下一步，是拿一条线上真实路径完成五问审计，保存一个可重放样本，再把发现的第一个断点修到可验证终态。

6.6 本章小结

对话记录说明 **Agent** 说过什么，工具结果说明内部组件声称什么，用户边缘证明确认用户真正关心的结果，执行凭证则保存从提议到边缘结果的完整链。五问审计把自然语言交互还原为可检查的生产运行，也把“**Agent** 看起来聪明”转换成“系统能够拥有状态、约束动作并证明结果”的工程标准。